

Full-Stack AWS Deployment Project

Docker | EC2 | RDS | S3 | Terraform | Load Balancer

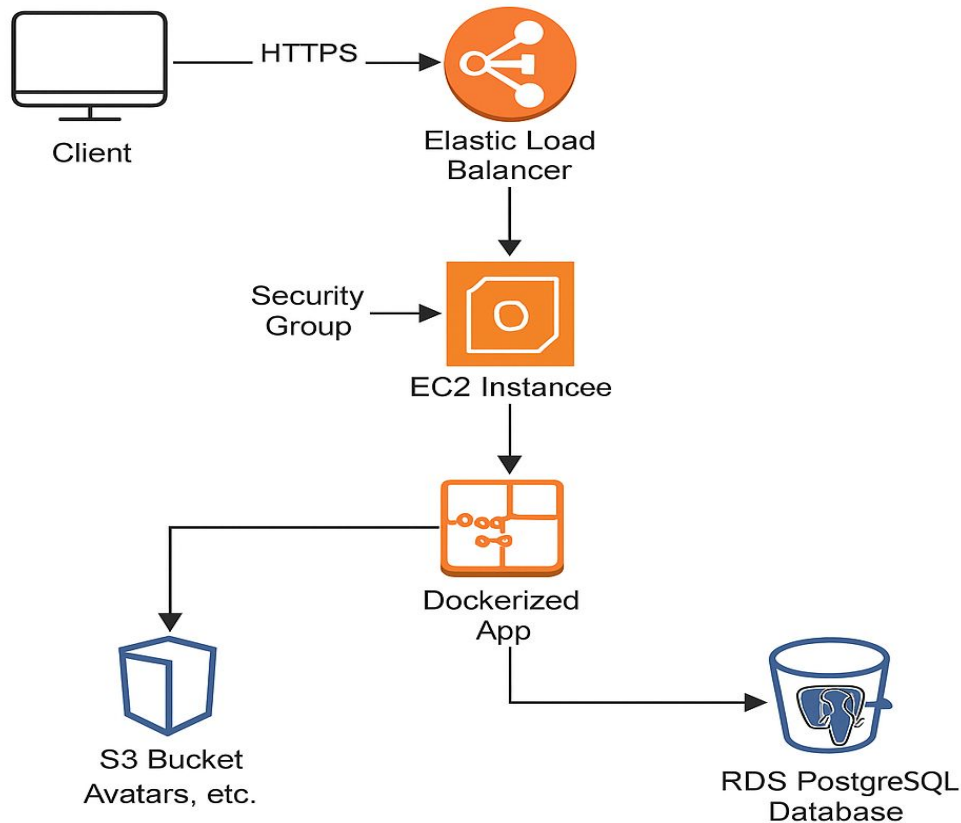
Presented by: Sumi Dhakal

Project Overview

End-to-end deployment of a cloud-based web application using AWS services:

- Dockerized backend and frontend
- EC2 hosting with Elastic Load Balancer
- RDS PostgreSQL for persistent database
- S3 for static content (user avatars)
- Infrastructure-as-Code using Terraform

Architecture Diagram



Key Features

- Dockerized full-stack app deployed on EC2
- Elastic Load Balancer to manage traffic
- PostgreSQL database hosted on RDS
- Media files served from S3
- Infrastructure provisioned using Terraform

Tools & Technologies

- AWS (EC2, RDS, S3, ELB)
- Docker, GitHub, Terraform
- PostgreSQL, AWS CLI
- Linux Server (Amazon Linux 2)
- Infrastructure Monitoring

Infrastructure Setup

Terraform IaC organized as follows:

-  infrastructure/
-  main.tf – EC2, RDS, S3 definitions
-  versions.tf – provider and Terraform version
-  terraform.tfstate – current state
-  terraform.tfstate.backup – backup of previous state

Infrastructure as Code (Terraform)

- Defined EC2, RDS, S3, and security groups in .tf files
- Used main.tf, versions.tf, and terraform.tfstate
- Automated resource provisioning and teardown
- Improved deployment consistency and reproducibility

Docker and Deployment

- Created a custom Dockerfile
- Built and tested container locally
- Deployed Dockerized app to EC2
- Connected securely to RDS and S3

Security Features

- Restricted access via Security Groups
- HTTPS traffic enforced using Load Balancer
- IAM roles used for secure access to S3
- RDS encrypted and not publicly accessible

Final Outcome & Learnings

- Fully deployed full-stack web app on AWS
- Secure, scalable, and production-ready environment
- Reusable infrastructure for future projects
- Strengthened portfolio with real-world cloud project